



Module 23-1

SystemVerilog Overview and Selected Language Constructs

cā dence®

This page does not contain notes.

Module Objectives

In this module, you

- Identify the purpose of SystemVerilog
- Analyze the segmentation of SystemVerilog constructs
- Examine two-state data types and their uses
- Use the new `logic` data type and analyze the relaxed rules for data types in SystemVerilog
- Use selected new SystemVerilog procedural statements and enhancements to existing constructs
- Use new synthesizable procedural blocks
- Examine the `inside` construct
- Use enumerated user-defined type declarations

This is only a very brief introduction to SystemVerilog!



This page does not contain notes.

What Is SystemVerilog?



SystemVerilog is a unified hardware design, specification, and verification language based on the Accellera SystemVerilog 3.1a extensions to the Verilog hardware description language.

It is a set of extensions to Verilog-2001, including the following:

- Added data types and relaxation of rules on existing data types
- Higher abstraction-level modeling features
- Language enhancements for synthesis
- Interfaces to model communication between design blocks
- Language features to enable verification methodologies
 - Assertions, constrained randomization, functional coverage
- Lightweight interface to C/C++ programs

Aim

- To define a concise, unified language for design and verification using a single simulation tool
 - Assuming tool support for enhanced testbench and verification features



SystemVerilog is the IEEE Std. 1800 “IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language”.

SystemVerilog includes the IEEE Std. 1364-2001 “IEEE Standard Verilog Hardware Description Language” by reference.

Several companies donated proprietary content to the Accellera EDA advocacy group, which integrated those Verilog extensions into SystemVerilog and championed the creation of the new IEEE standard.

Quick Reference Guide: SystemVerilog Segmentation



Verification Extensions			
Classes and Object-Oriented Design Dynamic Data Structures Scope and Class Based Constrained Randomization Intraprocess Synchronization Direct Programming Interface (DPI)		Assertions Coverage Clocking Blocks Program Blocks	<i>Increase Verification Efficiency!</i>
Design Subset			
Relaxed Datatype Rules Static Data Types Interfaces Specialized Procedures		Procedural Statements Connectivity Enhancements Convenience Features	<i>Express more with less code! Increase synthesis quality!</i>
Verilog-2001			
ANSI C style ports generate localparam constant functions	standard file I/O \$value\$plusargs `ifdef `elsif `line @*	(* attributes *) configurations memory part selects variable part select	multi dimensional arrays signed types automatic ** (power operator)
Verilog-1995			
modules parameters function/tasks always @ assign	\$finish \$fopen \$fclose \$display \$write \$monitor `define `ifdef `else `include `timescale	initial disable events wait # @ fork-join	wire reg integer real time packed arrays 2D memory modules parameters function/tasks always @ assign + = * / % >> <<

4

© Cadence Design Systems, Inc. All rights reserved.

cadence

Obviously, most SystemVerilog enhancements are for verification, have no hardware implementation and therefore are not synthesizable. These are the SystemVerilog verification extensions and include classes, coverage, randomization and assertions.

However, there are several SystemVerilog enhancements that are intended to fix common Verilog synthesis issues and which have hardware implementation-defined by the Language Reference Manual (LRM). This is the design subset of SystemVerilog intended for RTL code, and includes relaxed and enhanced data types, additional procedural statements and connectivity enhancements.

Obviously, there are many features that can be used in both verification and design. There is a large “grey area” of language features for which synthesizability and implementation are not defined by the LRM but depend on the synthesis tool you are using. There are even constructs that are definitely synthesizable with a known implementation in one form but vendor-specific in another. This can affect the portability of design code in that some constructs may be synthesizable in one tool, but not in another, or may be synthesizable, but implemented differently in different tools.

Normally, a company will make a decision as to whether portability and implementation are important and refine their existing RTL coding guidelines accordingly.

What Is a Datatype?



A datatype is a set of values (2-state or 4-state) that can be used to declare data objects or to define user-defined data types.

- The Verilog datatypes have 4-state values: (0, 1, Z, X).
- SystemVerilog adds 2-state value types based on **bit**:
 - Has values 0 and 1 only.
 - Direct replacements for **reg**, **logic** or **integer**.
 - Greater efficiency at higher-abstraction level modeling.

Newly introduced SV 2-state datatypes

Type	Description	Sign
bit	Single bit, Scalable to vector	Default unsigned
byte	8-bit vector or ASCII character	Default signed
shortint	16-bit vector	Default signed
int	32-bit vector	Default signed
longint	64-bit vector	Default signed



All Verilog data types except **real** are 4-state logic. 4-state logic is essential for gate-level modeling and for initialization at RTL, but simulating everything in 4-state logic can add significant performance overhead.

Therefore SystemVerilog adds a 2-state **bit** type, containing only 0 and 1 values, with the intention of enhancing simulation performance at the higher abstraction levels.

As with the 4-state register type **integer**, SystemVerilog defines a number of predefined **bit** types of various widths – **shortint** is 16 bits, **int** is 32 bits, and **longint** is 64 bits, on all platforms. Objects of the **bit** type are by default **unsigned**, and objects of the predefined types are, by default, **signed**.

SystemVerilog also defines a 32-bit floating-point type, **shortreal**, to complement the 64-bit Verilog **real** type.

What Is a 'logic'?



The keyword `logic` defines that the variable or net is a 4-state data type.

- The Verilog `reg` data type is potentially confusing:
 - It can synthesize to a net or register depending on the usage.
- SystemVerilog redefines register data types as variable (`var`).
- SystemVerilog also defines the 4-state type as `logic`.
 - `var logic` (variable of `logic` type) replaces `reg`.
 - The `var` keyword is optional.

Verilog

```
reg clock;
reg [15:0] b_reg;
reg [7:0] mem8x32 [0:31];
```

SystemVerilog

```
logic clock;
logic [15:0] b_reg;
logic [7:0] mem8x32 [0:31];
```

SystemVerilog changes the terminology for data types, but in a way that is backward compatible with Verilog.

Verilog has two forms of data type: net (`wire`) and register (`reg`). However, the term *register* is misleading. A synthesis tool may infer either combinational or sequential logic for a Verilog register type, depending on how it is used. Therefore, SystemVerilog uses the term *variable* (or `var`) to refer to Verilog register types.

SystemVerilog also defines the name `logic` for the basic 4-state Verilog type (0, 1, X, Z).

Therefore, the Verilog declaration `reg clk;` becomes `var logic clk;` in SystemVerilog. The default SystemVerilog data type for `logic` is variable, making the `var` keyword optional. Therefore the SystemVerilog form becomes `logic clk;` So `logic` becomes a direct replacement for `reg`.

You can also specify the `logic` type for net data types like `wire`. So Verilog `wire clk;` becomes `wire logic clk;` in SystemVerilog. However, SystemVerilog assumes that all net data types are `logic`, making the `logic` keyword optional. Therefore, the SystemVerilog form becomes `wire clk;`

Example: Usage of 2-State and 4-State Types

```
bit [7:0] twostate;
logic [7:0] fourstate;
...
twostate <= fourstate;
```

- 2-state variables initialize to 0
 - Hides failure to initialize design

- 4-state variables initialize to X
 - Exposes failure to initialize design

4-state to 2-state

0	0
1	1
Z	0
X	0



Avoid mixing 2-state
and 4-state types

Usage of 2-state
types in TestBench

```
bit [7:0] test_number;
bit [7:0] mem8x32 [0:31];
int i;
...
mem8x32[0] = test_number;
mem8x32[1] = i;
```

Although 2-state types are more efficient for simulation, they have some issues for RTL design.

Remember that the x and z values in 4-state logic are used for more than modeling unknown and tristate values. X is used for un-initialized logic (not yet assigned) and z for un-driven logic (declared but never assigned). Hence 4-state types always initialize to x at the start of the simulation, allowing the detection of un-initialized code.

2-state types initialize to 0 at the start of the simulation; hence un-initialized code cannot be so easily detected.

SystemVerilog allows 2- and 4-state types to be easily mixed, although this can be dangerous. If a 4-state type is assigned to a 2-state type, any x or z value is converted to 0. This can hide un-initialized or un-driven values. The conversion occurs with no warning or error, and so mixing 4-state and 2-state types is generally not a good idea.

Due to the initialization and conversion issues with 2-state types, they are rarely used in RTL code. However, they can be useful for testbench code and for the definition of the clock and reset variables which should not need x or z values.

Quick Reference Guide: Verilog Data Type Rules



- Verilog has strict data type rules:
 - **Variables** (registers) are assigned values in procedural blocks.
 - **Nets** are driven by continuous assignments, module inputs, module instance outputs, or primitive instances.
- These lead to the following connectivity characteristics:
 - Module inputs are always **nets**.
 - Module outputs are **variables** if driven by a procedural block, or **nets** in all other cases.
 - Connections to the input ports of a module instance are **variables if driven by a procedural block**, or **nets in all other cases**.
 - Connections to the output ports of a module instance are always **nets**.
 - Connections to bidirectional `inout` ports are always **nets**.



Remember the Verilog rules:

- You assign to register types (`integer`, `real`, `reg`, `time`) only in procedural blocks.
- You drive net types by continuous assignments or from outputs of modules or primitives.

This means that module input ports must connect internally to nets, module output ports must connect externally to nets, and module inout ports must connect both internally and externally to nets. Only module output ports can connect internally to variables.

The next page shows an example using the Verilog data type rules.

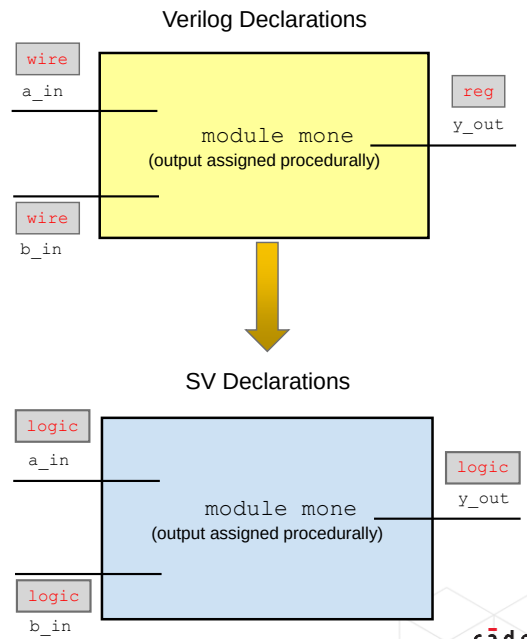
Relaxation of Datatype Rules



IEEE 1800-2012 6.3.1

SystemVerilog relaxes the rules for a **variable** by:

- Assigning a SystemVerilog **variable**:
 - In any number of `initial` or `always` blocks:
 - As in Verilog currently
 - From a single continuous assignment
 - From a single module `out` port
 - From a single primitive output
- Thus, you can declare most design signals to be **variable**.
 - As the `var` keyword is optional, you can simply declare most signals as type `logic`:
 - `logic my_var;`



9

© Cadence Design Systems, Inc. All rights reserved.

cadence

To ease confusion about when to use a Verilog register type and when to use a net type, SystemVerilog relaxes the data type rules. A SystemVerilog variable can be driven from a continuous assignment or a module or primitive output, as long as there is only **one** driver of that form. Therefore, in most situations, you can simply use the `logic` keyword instead of `reg` or `wire`.

The next page contains an example using the relaxed SystemVerilog data type rules.

Verilog Data Type Rules: Example

```
module mone(output reg y_out,
            input wire a_in, b_in);
  always @(a_in or b_in)
    y_out = a_in && b_in;
endmodule
```

y_out is **reg** in mone –
assigned in a procedural block

a_in, b_in are **wire** in mone
and mtwo – module inputs

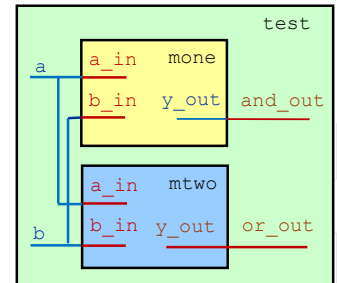
```
module mtwo(output wire y_out,
            input wire a_in, b_in);
  assign y_out = a_in || b_in;
endmodule
```

y_out is **wire** in mtwo – assigned
by a continuous assignment

```
module test;
  reg a, b;
  wire and_out, or_out;
  mone u1 (and_out, a, b);
  mtwo u2 (or_out, a, b);
  initial begin
    a = 0;
    b = 0;
    ...
  end
endmodule
```

and_out, or_out are
wire in module test –
instance outputs

a, b are **reg** in module test –
assigned in an initial
procedural block



10 © Cadence Design Systems, Inc. All rights reserved.

cadence

Here, we are using fully defined Verilog-2001 ANSI C syntax for module port declarations for clarity.

In both modules `mone` and `mtwo`, the input ports `a_in` and `b_in` must be declared as net data types.

In module `mone`, the output `y_out` must be declared as a register data type as it is assigned from the `always` procedural block.

In module `mtwo`, the output `y_out` must be declared as a net data type as it is driven from the continuous `assign` statement.

In module `test`, `a` and `b` must be declared as register data types as they are assigned from the `initial` procedural block, but `and_out` and `or_out` must be declared as net data types as they are driven from the instance output ports.

Therefore, a connection like `a -> a_in`, or `y_out -> and_out` actually changes data type as it crosses the module boundary.

Relaxed Data Type Rules: Example

```
module mone(output logic y_out,
            input logic a_in, b_in);
  always @(a_in or b_in)
    y_out = a_in && b_in;
endmodule
```

y_out is **logic** in mone –
assigned in a procedural block

a_in, b_in are **logic** in mone and
mtwo – connected to single module inputs

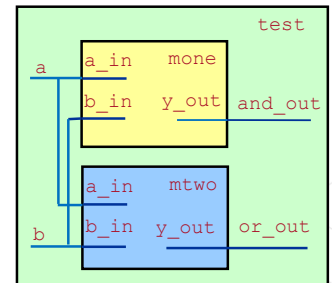
```
module mtwo(output logic y_out,
            input logic a_in, b_in);
  assign y_out = a_in || b_in;
endmodule
```

y_out is **logic** in mtwo – assigned
by a single continuous assignment

```
module test;
  logic a, b;
  logic and_out, or_out;
  mone u1 (and_out, a, b);
  mtwo u2 (or_out, a, b);
  initial begin
    a = 0;
    b = 0;
    ...
  end
endmodule
```

and_out, or_out are
logic in module test
– single instance outputs

a, b are **logic** in module test –
assigned in an initial
procedural block



11

© Cadence Design Systems, Inc. All rights reserved.

cadence

Using the relaxed data type rules of SystemVerilog, our example becomes easier to read and write.

In both modules mone and mtwo, the input ports a_in and b_in can now be declared as register data types (**logic**) as they are both driven from single input ports of the module.

In module mtwo, the output y_out can now be declared as a register data type (**logic**) as it is driven from a single continuous assign statement.

In module test, and_out and or_out can now be declared as register data types (**logic**) as they are driven from single output ports of the module instances.

We can simply use logic everywhere in this example.

Therefore, a connection like a-> a_in or y_out -> and_out no longer changes data type as it crosses the module boundary.

How to Overcome Variable Assignment Restrictions



Although SystemVerilog relaxes the rules on data type declaration, there are still some restrictions on assignments to variables. These restrictions aim to prevent multiple drivers on a single variable.

- You cannot combine procedural assignments with continuous assignments or module output drivers on the same variable.
- You cannot have multiple continuous assignments or multiple output ports drive the same variable.
- Only net types can have multiple drivers.

```
module test;
  logic in1, in2, in3;
  logic op;
  mone u1 (op, in1, in2);
  mtwo u2 (op, in1, in3);
endmodule
```



Error
Multiple drivers
on logic op

```
module test;
  logic in1, in2, in3;
  wire op;
  mone u1 (op, in1, in2);
  mtwo u2 (op, in1, in3);
endmodule
```



Although SystemVerilog relaxes the rules on data type declaration, there are still some restrictions on assignment to variables. These restrictions aim to prevent multiple drivers on a single variable.

If a variable is assigned from a procedural block, then it cannot also be driven from a continuous assign statement or from a module output port.

A variable cannot be driven by multiple assign statements or multiple module output ports or by a combination of assign and port drivers.

Only net data types (like wire) are allowed to have multiple drivers.

The intention is that signals with single drivers are declared as variables, and net types are reserved solely for connections with multiple drivers only.

However, you still have the problem of driving a variable from multiple procedural blocks. SystemVerilog has a different solution for this problem.

Assignments to a Variable from Multiple Procedures



Verilog allows assignments to a variable from multiple procedures.
How do you avoid this in SystemVerilog?

- Special procedural blocks allow only a single driver to variables.
 - Also give much greater clarity to RTL code
 - `always_ff`, `always_comb`, `always_latch`
 - See later

```
logic op;  
  
always @(sel, a, b)  
  if (sel)  
    op = a;  
  else  
    op = b;  
  
always @(sel2, c, d)  
  if (sel2)  
    op = c;  
  else  
    op = d;
```



```
logic op;  
  
always_comb  
  if (sel)  
    op = a;  
  else  
    op = b;  
  
always_comb  
  if (sel2)  
    op = c;  
  else  
    op = d;
```



Error

`always_comb` allows
only a single driver
on logic `op`.

SystemVerilog has a set of new procedural blocks for RTL code. These allow the designer to clearly identify how they intend the procedural block to be synthesized. For example, `always_comb` is an `always` block that should synthesize to combinational logic.

Another feature of these new procedural blocks is that they enforce a “single driver per variable” policy. If a variable is assigned from one of these procedural blocks, then it is a compilation error to drive the same variable from any other procedural block, or `assign` statement or module output.

What Are Time Literals?



Time Literals are numbers written in integer or fixed-point format, followed without a space by a time unit (`fs ps ns us ms s`).

- SystemVerilog provides a time literal construct that:
 - Explicitly specifies time values.
 - Allows more control over time literals.
- Format:
 - Integer or fixed-point value.
 - Immediately followed by time unit:
 - With no space between the value and unit.
 - Having units of (`fs ps ns us ms s`).
 - Scaled to the current time unit.
 - Rounded to the current precision.
 - Also having a `1step` delay value:
 - One simulation precision unit.

`42ns`
`3.14ps`
`1step`

Only `1step` is allowed
Never `2step`, etc.



You write a time literal as an integer or fixed-point number, followed immediately by a time unit (`fs ps ns us ms s`) without an intervening space character.

The `1step` delay refers to the simulation step, that is, the simulation precision.

Time literals allow delay values to be explicitly defined and so isolated from the current time unit, traditionally specified with a ``timescale` compiler directive. However, SystemVerilog still rounds the time literal to the current time precision. This implies that you must have a current time unit and precision specified for the scope where the time literal appears.

Example: Time Units in Assignments

```
`timescale 1ns / 100ps      // time unit 1ns, precision 100ps
module testmodule;
    logic avar, sel, bvar, cvar, clk;

    initial
    begin
        #20ns sel = 0;          // blocking assign
        #5.18ns sel = 1;        // here rounded to 5.2ns
        bvar = #1step cvar;      // blocking assign using 1step
    end

    always @(posedge clk)
        #5.1ns avar <= avar + 1; // nonblocking assign
    endmodule
```

In this example, #1step
advances simulation for
100ps

You can use a time literal or 1step anywhere you would use a delay

You can use a time literal anywhere you would use a unitless integer or real number to represent time.

This example uses time literals to specify procedural and intra-assignment delays.

Although the time literal isolates a delay value from the current time unit, the delay is still scaled by the current time precision. In the example above, the #5.18ns delay is scaled to #5.2ns as the precision is 100ps (0.1ns).



timeunit and timeprecision



IEEE 1800-2012 22.7

- Equivalent to ``timescale`, but without file order compilation dependency.
- `timeunit` and `timeprecision` are:
 - Declarations, not directives.
 - Placed inside the module.
 - Must be the first statements of the module.
 - Only visible in the design element where they are declared and any nested units.
- A default `-timescale` command line option is still the best approach

```
module testbench;
  timeunit 1ns;
  timeprecision 100ps;
  logic status;
  initial begin
    #10ns status = 1'b1;
    #10 status = 1'b0;
  end
endmodule
```

```
module testbench2;
  timeunit 1ns/100ps;
  logic status;

  initial begin
    #10ns status = 1'b1;
    #10 status = 1'b0;
  end
endmodule
```

16 © Cadence Design Systems, Inc. All rights reserved.

cadence

The `timeunit` and `timeprecision` declarations are equivalent to the `timescale` compiler directive (``timescale`), but without the file-order dependency of compiler directives. `timeunit` and `timeprecision` must be the first declarations in the module or design element. The declarations are visible only within that unit declaration and within any further nested declarations.

Syntax: `timeunit <literal><units> ;`
`timeprecision <literal><units> ;`

Where:

`literal`::= 1, 10, 100

`units`::= s ms us ns ps fs step

- `step` advances time by the simulation time precision.

The current time unit is defined using the following highest to lowest precedence:

- 1st – The `timeunit` of the current interface, module, package or program definition.
- 2nd – The `timeunit` inherited from the enclosing interface or module definition.
- 3rd – The `timescale` (``timescale`) directive of the current compilation unit.
- 4th – The `timeunit` declaration of the current compilation unit.
- 5th – The implementation-specific default time unit.

Variable Declarations in `for` Loop Statements



IEEE 1800-2012 12.7.1

You can declare a `for` loop variable within the `for` statement.

- Variables are visible only in the loop.
 - So the same identifier can be safely used in multiple (un-nested) loops.
- Good use of 2-state data types as loop variables
- Dedicated increment (`++`) and decrement (`--`) operators also useful

Verilog

```
// loop variable declared outside loop
integer i;
initial begin
  for (i=0; i<10; i=i+1)
    ...
```

ISSUE: Two loops share single variable

```
initial begin
  for (i=0; i<8; i=i+1)
    ...
```

int: Good use for 2-state types

SystemVerilog

```
initial begin
  // loop variable declared in loop
  for (int i=0; i<10; i++)
    ...
initial begin
  for (int i=0; i<8; i++)
    ...
```

Variable `i` visible to `for` loop only

Increment operators `++`

Two loops, two independent variables

One benefit of allowing variables in unnamed blocks is that you can now declare a `for` loop variable within the `for` statement, removing the need to separately declare the variable outside the loop. This has the added benefit of avoiding issues where a single loop variable is accidentally or deliberately used in multiple, overlapping loops.

In fact, you can declare multiple variables in the loop, separately initialize each variable, and separately reassign each variable. Thus, SystemVerilog `for` loops can be a function of more than one loop variable.

foreach Loop



IEEE 1800-2012 12.7.3

This loop iterates over all the elements of an array.

Loop variable characteristics:

- Does not have to be declared.
- Are read only.
- Only visible inside loop.

Use multiple loop variables for multidimensional arrays.

- Equivalent to nested loops.

Useful for initializing and array processing.

Equivalent SV code

```
for (int i=7; i>=0; i=i-1)
  intarr[i] = 7-i;
```

Iterates left bound to right bound

```
int intarr [7:0];
...
foreach (intarr [i])
  intarr[i] = 7-i;
```

```
int arr2d [7:0] [2:0];
...
foreach (arr2d [k, l])
  arr2d[k][l] = k*l;
```

Equivalent SV code

```
for (int k=7; k>=0; k=k-1)
  for (int l=2; l>=0; l=l-1)
    arr2d[k][l] = k*l;
```

Two loop variables for a 2D array creates nested loops

The SystemVerilog foreach loop iterates over all the elements of an array. It is equivalent to a for loop, which iterates over the full width of the array from left bound to right bound. This construct is useful for simple loop-based array initialization or processing.

The foreach loop variable is automatically declared (and typed) as part of the foreach statement. As a local variable, it is visible only within the foreach statement and any nested blocks. In a change to normal Verilog behavior, this variable cannot be written but only read.

If the array is multidimensional, you can declare a loop variable for each dimension. This creates nested loops. Loop variables are mapped to dimensions in order of cardinality (see the *IEEE1800 Language Reference Manual*). In the example above, k iterates from 7 to 0, and l from 2 to 0. Also, higher cardinality dimensions change more rapidly, so in the example above, k is set at 7 while l iterates from 2 to 0, then k decrements to 6 while l changes, etc.

Syntax:

```
foreach ( array_identifier [ loop_variables ] )
  statement(s)
```



RTL Coding Guidelines for Synthesis Using Verilog (Review)

Hardware	Rules	Example
Combinational logic	An always block with: - Complete (or wildcard) sensitivity list - Complete assignment to every output in every path	<pre>always @(a or b or sel) if (sel == 1) op3 = a; else op3 = b;</pre>
Latch	An always block with: - Complete (or wildcard) sensitivity list - Incomplete assignment to outputs	<pre>always @(sel or a) if (sel) op2 <= a;</pre>
Register	An always block with: - Sensitivity list with edge expression on clock (and asynchronous reset if used) - Incomplete assignment to outputs	<pre>always @(posedge clk or posedge rst) if (rst) op1 <= 8'h00; else op1 <= data_in;</pre>

1. Synthesizability is controlled by sensitivity list and coding style.

2. Simulator has no idea what user intends.



19

© Cadence Design Systems, Inc. All rights reserved.

cadence

A Verilog `always` block can synthesize to combinational, latched, or sequential logic, depending upon the sensitivity list and your coding style. The simulator does not know what you intend, so it cannot verify that your block matches your intentions.

SystemVerilog adds implementation-specific procedural blocks (`always_comb`, `always_latch`, `always_ff`). These blocks reduce design ambiguity by clearly indicating the hardware intent for a procedural block. Simulation, formal checking, linting, synthesis, equivalence checking and other downstream tools can have a consistent specification.

Combinational Blocks with `always_comb`



IEEE 1800-2012 9.2.2.2

SystemVerilog adds a specialized procedural block for modeling combinational logic:

- Implied, complete sensitivity list.
- Any variable assigned in an `always_comb` cannot be assigned by another procedure.
- Cannot contain further blocking timing or event control.
- Automatically executed once at time 0 without waiting for an event.
 - After all `initial` and `always` blocks have executed.
 - Ensures outputs are consistent with inputs.
- Tools may issue warnings if the block does not infer combinational logic.

```
always_comb
  if (sel == 1)
    op = a;
  else
    op = b;
```

```
logic op;
always_comb
  if (sel)
    op = a;
  else
    op = b;

always_comb
  if (sel2)
    op = c;
  else
    op = d;
```

 Error

20 © Cadence Design Systems, Inc. All rights reserved.

cadence

The `always_comb` procedural block provides special functionality for combinational logic:

- It infers a sensitivity list that includes every variable read by the procedural block.
- It prohibits its assigned variables from being assigned in any other procedural block. (With these processes, it is illegal for a variable to be written to by more than one process, even when these processes are in different modules, interfaces or test programs)
- It cannot contain any embedded timing or event control.
- It is initially run once at time zero after all `initial` and `always` blocks have run to suspension. This is to ensure that the outputs of the block are consistent with any changes to the inputs.

Normally, an `always` block will be executed only at time 0 if there is an event on a signal in its sensitivity list. In addition, the execution order of `always` and `initial` blocks is indeterminate. A testbench `initial` block may apply an initial value to a design input that does not trigger an event (e.g., assignment of 0 to a 2-state type). Therefore, the outputs of a combinational block may not be consistent with the input values at time 0. Use of an `always_comb` fixes this initialization inconsistency by automatically executing the `always_comb` after `initial` and `always` blocks at time 0.

Software tools can perform additional checks to warn whether behavior within an `always_comb` procedural block does not represent combinational logic, e.g., whether it infers latches.

Comparing Constructs `always @*` and `always_comb`



`always @*`

- Verilog-2001 event control feature

- Can include timing and additional event controls.
- Can assign to variables which are assigned elsewhere.
- Triggered at time 0 with other blocks only if event on sensitivity list.
- Only sensitive to changes in the arguments of a called function.

```
always @*  
  if (sel == 1)  
    op = a;  
  else  
    op = b;
```



`always_comb`

- SystemVerilog procedural block

- Cannot contain any timing or event controls.
- Cannot assign to variables which are assigned elsewhere.
- Automatically triggered at time 0 after `always` and `initial` blocks.
- Sensitive to changes in any input to a called function.

```
always_comb  
  if (sel == 1)  
    op = a;  
  else  
    op = b;
```

Verilog-2001 added a wildcard sensitivity list for combinational `always` procedures. However, this is only an enhancement for the event expression; it does not prevent the `always` block from containing other event controls or timing, from driving variables that are assigned elsewhere nor trigger execution of the block at time 0.

Also, the wildcard sensitivity has a limitation. If the `always` block contains a function call, then only the arguments of the function are added to the sensitivity list. If the function reads a variable that is not passed via the argument list (i.e., reads the variable by side effects), then that variable is not included in the wildcard sensitivity list.

The SystemVerilog `always_comb` block includes the extra features which prevent additional event or delay expressions in the block; it prevents multiple drivers on assigned variables and triggers the execution of the block at the start of simulation. In addition, the complete sensitivity list of `always_comb` includes all variables read by a function call, including those read by side effects.

Therefore, in every way, the `always_comb` block is far better for combinational synthesis code than the `always` block.

Latch Blocks with `always_latch`



IEEE 1800-2012 9.2.2.3

SystemVerilog adds a specialized procedural block for modeling latched logic:

- Implied, complete sensitivity list.
- Variables assigned in an `always_latch` cannot be assigned by another procedure.
- Cannot contain further block timing or event control.
- Automatically executed once at time 0 without waiting for an event:
 - After all `initial` and `always` blocks have executed.
 - Ensuring outputs are consistent with inputs.
- Tools can issue warnings if the block does not infer latched logic.

```
always_latch
if (gate == 1)
  op <= a;
```

```
always_latch
if (en1)
  op <= c;

always_latch
if (en2)
  op <= d;
```



22

© Cadence Design Systems, Inc. All rights reserved.



The `always latch` (`always_latch`) procedural block provides special functionality for latched logic, similar to an `always_comb` block.

- It infers a sensitivity list that includes every variable read by the procedural block.
- It prohibits its assigned variables from being assigned in any other procedural block.
- It cannot contain any embedded timing or event control.
- It is initially run once at time zero after all `initial` and `always` blocks have run to suspension. This is to ensure that the outputs of the block are consistent with any changes to the inputs.

However, the best feature of the `always_latch` is that it clearly documents that a designer intends to create latched logic.

Software tools can perform additional checks to warn whether the behavior within an `always_latch` procedural block does not represent latched logic.

Register Blocks with `always_ff`



IEEE 1800-2012 9.2.2.4

SystemVerilog adds a specialized procedural block for modeling registered logic:

- Variables assigned in `always_ff` cannot be assigned by another procedure.
- Contains one and only one event control.
- Cannot contain any block timing.
- Tools may issue warnings if the block does not infer registered logic.

```
always_ff @(posedge clk or posedge rst)
  if (rst)
    op <= 1'b1;
  else
    op <= ip;
```

23 © Cadence Design Systems, Inc. All rights reserved.



The `always` flip-flop (`always_ff`) procedural block requires one and only one event control preceding the block body.

- It prohibits its assigned variables from being assigned in any other procedural block.
- It cannot contain any additional embedded timing or event control.

Software tools can perform additional checks to warn whether the behavior within an `always_ff` procedural block does not represent sequential logic.

Set Membership Operator



IEEE 1800-2012 11.4.13

- `inside` does a comparison:
 - True if expression value is contained within a value list.
- List values can be:
 - Variables (including arrays)
 - Literal values including ranges.
 - Literal values with wildcards.
 - Overlapping
 - A mixture of any of the above

```
if ( a inside {2'b01, 2'b10} )
...
// equivalent to:
if ((a==2'b01) || (a==2'b10))
```

```
int bar [1:0] = '{5,6};
if ( a inside {bar, [0:2]} )
...
// equivalent to:
if (a inside {bar[1], bar[0], 0, 1, 2})
```

```
if ( a inside {2'b0?} )
...
// equivalent to
if ( a inside {2'b00, 2'b01, 2'b0X, 2'b0Z})
```

24

© Cadence Design Systems, Inc. All rights reserved.

cadence

The set membership (`inside`) operator performs a comparison between a left-side expression and a right-side list of values. The right-side list can utilize wildcards, value ranges and the values can overlap.

Range values must be expressed in the form `[lower_bound : higher_bound]`. Either bound can be `$` indicating the minimum or maximum value for the expression.

For nonintegral expressions, SystemVerilog performs an equality match operation, which returns 1 or 0.

For integral expressions, SystemVerilog performs a wild equality match operation, which is an asymmetric match. Therefore, the result can be 0, 1 or unknown.

Syntax:

- **expression inside { value_range { , value_range } }**
- **value_range ::= expression | [expression : expression]**

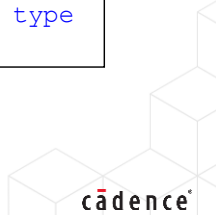
User-Defined Types

- You can declare and name a type using `typedef`:
 - Verilog standard types
 - SystemVerilog user-defined types
- The type name can be used anywhere, the type declaration is legal:
 - For example: variable or port declaration
- You must declare the `typedef` before the name is used.
- The declaration must be visible in the scope where the name is used.
- An un-named type is known as an anonymous type.

```
typedef logic [7:0] vec8_t;  
  
input vec8_t op1;  
vec8_t busa, busb;  
function vec8_t checksum;  
    ...
```

```
// typed (named) type  
typedef logic [15:0] vec16_t;  
vec16_t wordvar;
```

```
// anonymous (un-named) type  
logic [16:0] wordvar;
```



A `typedef` declares and names a user-defined type, allowing the name to be used in the declaration of variables, ports and other types. You must declare a `typedef` before you use the name, either:

- Inside any declarative scope (for example, a module), which makes it visible only within that scope, and any nested declarative scopes.
- In the compilation unit. (See the Hierarchy and Connectivity module for more details.)

Or

- Inside a package, which makes it visible in all design elements that import the package. (See the Hierarchy and Connectivity module for more details.)

What Is an Enumerated Type?



An enumerated type is a set of integral named constants.

- It is a type with user-defined values:
 - Value names must be unique in the current scope.
- Declared with the `enum` keyword:
 - Named or anonymous.
- Enumerated types are strongly typed.
- Should only be directly assigned:
 - A named value of the type.
 - A variable of the same type.
- Naming the type (`typedef`) permits typecasting:
 - `type_name' (value)`

```
// named enum type
typedef enum {idle, start,
              pause, done} state_t;
state_t state, next_state;

// anonymous enum type
enum {idle, start,
      pause, done} astate;

...
state = idle;
state = next_state;
state = 2'b00;
state = 2;
state = state_t'(2);
...
```

Legal

Legal

Direct Assignment of integral value to enum variable is not legal, but generally allowed (with warnings)

An enumerate type is a user-defined type where the user defines the value set of the type. A SystemVerilog enumerate type is declared with the keyword `enum` followed by a list of values for the type. The value names must be unique in whichever scope the enumerate type is used. A common policy is to use uppercase names for values to ensure uniqueness.

Enumerate type declarations can be typed (with `typedef`) or anonymous. There are significant benefits to naming the enumerate type as we shall see.

SystemVerilog enumerate types are strongly typed. An enumerate variable can be assigned only from a variable of the same type or from an enumeration value of its type.

To assign any other value requires a static cast to the enumerated type. A cast can only be made using the name of the enumerate type. Casting the value to the enumerated type does not check the validity of the value (see following slides).

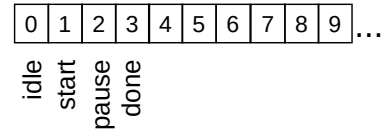
Traditionally, Verilog is not a strongly typed language. Therefore some compilers allow the direct assignment of an integral value to an enumerated type variable, but generate a warning.

Tip: Always use a `typedef` to declare your enumerate types.

Enumerated Type Values

- Default base type of an enum is `int`.
- Default values increment from zero in declaration order.
- An enumerate variable can hold any value from the base type:
 - Even if there is no matching enumerate value name.
 - Casting does not check value.
- An enumerated type is the base type with mnemonics for specific values.
- An enumerate variable in an expression is replaced by its base type value.

```
typedef enum {idle, start,
              pause, done} state_t;
// idle=0; start=1
// pause=2; done=3
```



```
state_t state;
int aint;
initial begin
  state = state_t'(2); // pause
  state = state_t'(8); // no name

  aint = state * 2;    // 16
  ...
```

27 © Cadence Design Systems, Inc. All rights reserved.

cadence

By default, the base type of an enumerate type is `int`.

By default, value encodings for enumerated types are controlled by the order in which they are declared. The first value is represented by 0, second by 1 and so on.

An enumerate type can be considered as its base type (default `int`) with names (mnemonics) for specific values. `state` can be assigned (via casting) any `int` value.

Therefore:

```
state = state_t'(8);
```

is legal, even though there is no named value in the enumerate declaration corresponding to the integral value 8.

When you use an enumerate variable in an expression, SystemVerilog automatically converts the enumeration value to its base integral value and uses the integral value to evaluate the expression. If you wish to assign the expression result back to the enumerate variable, you must use an explicit cast:

```
state = state_t'(state * 2);
```

How to Assign Enumerate Value Encodings Explicitly



Explicit encoding can be included in an enumerated type declaration, either typed or anonymous which allows one-hot, grey or other encodings to be defined for enumerated values.

1. You can define explicit encoding for values:

- Allows one-hot coding, etc.

```
typedef enum {idle = 1,
              start = 2,
              pause = 4,
              done = 8} state_t;
```

0	1	2	3	4	5	6	7	8	9	...
	idle	start		pause				done		

2. You can mix explicit and implicit value encoding:

- Implicit values increment from previous explicit value.

```
enum {S0=0, S1, S2=6, S3} encs;
```

0	1	2	3	4	5	6	7	8	9	...
S0	S1					S2	S3			

3. Value encodings must be unique:

- Compilation error is reported if encodings overlap.



Error

```
enum {P0=0, P1, P2=1, P3} bad_encs;
```

P2 duplicates P1 encoding

Explicit encoding can be included in an enumerated type declaration, either typed or anonymous. This allows one-hot, grey or other encodings to be defined for enumerated values.

For 2-state types like the default `int` base type, you can use only the binary values (no Z or X).

You can mix explicit and implicit value encoding in the same enumerated type declaration. For any value without an explicit encoding, the encoding is simply an increment of the encoding for the previous enumerate value; hence for the variable `encs`, `S1` has the encoding 1, and `S3` = 7.

It is a compilation error if two or more values have the same encoding; e.g., for variable `badencs`, both `P1` and `P2` have an encoding of 1, `P1` by implicitly following the encoding for `P0`, and `P2` by explicit declaration.

Note: For these encodings to be reflected in hardware, the synthesis tool must support explicit encoding. If, for example, you want to optimize the encoding of a state variable, it may be better to allow the synthesis tool to choose its own encoding based on your synthesis goals.

Assigning Enumerate Base Type Explicitly



IEEE 1800-2012 6.19

- You can specify the enumeration base type:
 - Base type `int` by default
- This allows you to constrain the enumerate value range.
- The default value encoding will match the specified base type.
- Explicit value encoding must match type and (if explicit) length.



An explicit base type makes enumerates easier and safer to use.

Enum Declaration with **explicit base type** and **explicit encoding**

```
typedef enum bit[1:0]
{idle, start, pause, done} base_t;
// idle = 2'b00, start = 2'b01
// pause = 2'b10, done = 2'b11
```

00	01	10	11
idle	start	pause	done

Enum Declaration with **explicit base type** and **implicit encoding**

```
typedef enum bit[2:0]
{idle = 3'b000,
 start = 3'b001,
 pause = 3'b010,
 done = 3'b100} onehot0_t;
onehot0_t state;
...
state <= onehot0_t'(3'b001);
```

29

© Cadence Design Systems, Inc. All rights reserved.

cadence

The base type of an enumerated type defaults to `int`. As we have seen, this means a default enumerate variable can be assigned any value in the `int` range (32 bits).

You can define an explicit base type for the enumerate. This allows you to constrain the size of the base type and hence limit the range of values that can be assigned into the type by casting.

You can declare base types of the atomic integer types (`byte`, `shortint`, `int`, `longint`, `integer`, `time`) or integer vector types (`bit`, `logic`, `reg`).

If you define an explicit base type, then any explicit value encodings must match the base type and size. In the example, for type `onehot0_t`, as the base type is of size 3, the explicit encodings must all be sized to length 3. Truncation and padding assignment rules do not apply when defining explicit encodings. This is to ensure every value has a unique encoding.

Enumerated Base Type vs. Enum Initial Value

- The initial value of an `enum` variable depends on its base type:
 - 0 for default `int` type and any explicit 2-state type.
 - x for an explicit 4-state type.
- You can explicitly encode a value with x:
 - Typically, to indicate an illegal value.

```
enum bit[2:0]{BONE, BTWO} var_b;
```

```
initial begin
  $display("value %b", var_b);
  $display("name %s", var_b.name());
```

```
value 000
name BONE
```

```
enum logic[2:0] {LONE, LTWO} var_l;
```

```
initial begin
  $display("value %b", var_l);
  $display("name %s", var_l.name());
```

```
value xxx
name
```

Empty string

```
enum logic[2:0] {LX=3'bX, LONE=3'b001, ...} var_l;
```

```
initial begin
  $display("value %b", var_l);
  $display("name %s", var_l.name());
```

```
value xxx
name LX
```

cadence

The initial value of an enumerate variable depends on the base type of the enumerated declaration.

- For the default `int` base type, the initial value is 0, which would match the default encoding of the first enumeration value. Therefore, by default, the variable initializes to the first enumerate value.
- For any explicit 2-state type (e.g., `bit` array), the initial value is 0, which would match the default encoding of the first enumeration value.
- For an explicit 4-state base type (e.g., `logic` array), the initial value is all bits unknown ('x'), which would match only an explicitly encoded value.

In the second example, the initial value of enumerate `var_l` is 3'bxxx. As this value does not match the encoding for any of the declared values, the `name()` method returns an empty string.

With a 4-state explicit base type, you can encode a value with unknowns ('x'). This could be used, for example, to indicate that the variable is invalid and detect sampling problems. However, an explicit unknown value will cause problems if the variable is intended to be synthesized.

Quick Reference Guide: Enumerated Type Access Methods



Method	Description
first()	Returns first value
last()	Returns last value
next(N)	Returns next Nth value from current
prev(N)	Returns previous Nth value from current
num()	Returns number of values
name()	Returns string equivalent of value

```
typedef enum {SA, SB, SC, SD} state_t;
state_t st = st.first();
...
repeat(5) begin
    $display ("%s = %0d", st.name(), st);
    if (st == st.last())
        $display ("-----");
    st = st.next();
end
...
```

Iterates over defined values only

SA = 0
SB = 1
SC = 2
SD = 3

SA = 0

```
state_t op;
...
op = op.first();
repeat(op.num())
begin
    ...
    op = op.next();
end
...
```

Iterate over every value of an enumerate

This page does not contain notes.

Module Summary

SystemVerilog has a lot of new content, including:

- Nearly 100 new keywords

SystemVerilog is a substantial upgrade on Verilog:

- Based heavily on Verilog-2001

There is a huge learning curve for both designers and tool vendors.

- Parts of the language are still being refined
- Tools may not support some language constructs
- Most users will adopt SystemVerilog incrementally



Many design teams have not yet fully adopted the Verilog-2001 standard. As SystemVerilog is based on Verilog-2001, such teams will need to learn about Verilog-2001 as well as SystemVerilog.

Having many new keywords means that existing Verilog designs may not compile with SystemVerilog compilers. For example, if you have a signal named *do*, then your code will not compile in SystemVerilog because *do* is a SystemVerilog keyword.

However, SystemVerilog does not take features away from Verilog users. Verilog-2001 features are still available and work the same as earlier.